山东大学 计算机科学与技术 学院

机器学习 课程实验报告

学号：202200130077	姓名： 薛润泽	邮箱：

实验题目：RecSys 挑战赛—新闻文章推荐预测

实验过程：

（记录实验过程、遇到的问题和实验结果。可以适当配以关键代码辅助说明，但不要大段贴代码。）

1、RecSys 挑战赛内容

本次挑战赛要求建模出预测“用户从特定印象看到的文章列表中点击哪一篇文章”的模型。建模中可以运用到的信息有用户的行为（存储在 behaviors.parquet 中，主要包括用户的个人信息和用户与文章的交互信息等），用户的点击历史（存储在 history.parquet 中，主要包括用户身份，文章阅读状态等），文章内部信息（存储在 articles.parquet 中，主要包括文章主题内容等）。

2、实验思路

因为本次实验难度较大且给出了实例代码，所以本次实验的主要目的为修改实例代码，使得代码可以正常运行，从而预测用户点击情况。

2、实验代码模型简介

实验模型为 NRMS(Neural News Recommendation with Multi-Head Self-Attention）多头自注意神经新闻推荐模型。从一个新闻标题中学习新闻的表示，或从一个用户浏览的历史新闻中学习用户的表示。关键一点在于捕捉单词与单词之间、新闻与新闻之间的相关性。但是传统 CNN 卷积神经网络的缺点在于不能将文本距离过远的单词之间的相关性考虑在内。而 NRMS 通过自注意的方法，可以综合考虑任意两个单词之间的联系，弥补了传统 CNN 卷积神经网络的缺点。NRMS 在学习每一个单词的表示时，都会利用到标题中的所有单词，这是自注意力机制的体现。

3、示例代码分析与修改

比赛在文件材料的 00_quick_start 文件夹中给出了大量示例代码。我采用的示例代码为 nrms_ebnerd.ipynb。该示例代码基本完成了模型训练，但是该实例代码无法完成对 test 测试集的预测，因为代码中并没有引入 test 数据集。对于测试方面，nrms_ebnerd.ipynb 将 validation 验证集当做了测试集进行了输出。在代码修改方面，将输入的 validation 替换为测试集 test。

对于建立多头自注意神经新闻模型方面，通过导入相关库构建即可完成。

（1）本段代码完成了对功能库的导入。

Load functionality

```
from transformers import AutoTokenizer, AutoModel
from pathlib import Path
import tensorflow as tf
import polars as pl
import pandas as pd
from ebrec.utils._constants import (
    DEFAULT_HISTORY_ARTICLE_ID_COL,
    DEFAULT_CLICKED_ARTICLES_COL,
    DEFAULT_INVIEW_ARTICLES_COL,
    DEFAULT_IMPRESSION_ID_COL,
    DEFAULT_SUBTITLE_COL,
    DEFAULT_LABELS_COL,
    DEFAULT_TITLE_COL,
    DEFAULT_USER_COL,
)

from ebrec.utils._behaviors import (
    create_binary_labels_column,
    sampling_strategy_wu2019,
    add_known_user_column,
    add_prediction_scores,
    truncate_history,
)
```

（2）本段代码导入训练集，需要注意的是，在这里只需要导入 history.parquet 文件与 behaviors.parquet 文件即可，article.parquet 文件没有使用到。

而且，在这里导入的是 small 大小的数据集。通过使用 small 数据集进行训练。

Load dataset

```
def ebnerd_from_path(path: Path, history_size: int = 30) -> pl.DataFrame:
    """
    Load ebnerd - function
    """
    df_history = (
        pl.scan_parquet(path.joinpath("history.parquet"))
        .select(DEFAULT_USER_COL, DEFAULT_HISTORY_ARTICLE_ID_COL)
        .pipe(
            truncate_history,
            column=DEFAULT_HISTORY_ARTICLE_ID_COL,
            history_size=history_size,
            padding_value=0,
            enable_warning=False,
        )
    )
    df_behaviors = (
        pl.scan_parquet(path.joinpath("behaviors.parquet"))
        .collect()
        .pipe(
            slice_join_dataframes,
            df2=df_history.collect(),
            on=DEFAULT_USER_COL,
            how="left",
        )
```

（3）本段代码用以生成存储预测信息的标签数组，同时创建训练数据集 df_train 和验证数据集 df_validation（实际上没有使用到 df_validation）。

Generate labels

We sample a few just to get started. For testset we just make up a dummy column with 0 and 1 - this is not the true labels.

```
[3]
PATH = Path("/root/autodl-tmp/ebnerd-data")
DATASPLIT = "ebnerd_small"
```

Python

In this example we sample the dataset, just to keep it smaller. Also, one can simply add the testset similary to the validation.

```
COLUMNS = [
    DEFAULT_USER_COL,
    DEFAULT_HISTORY_ARTICLE_ID_COL,
    DEFAULT_INVIEW_ARTICLES_COL,
    DEFAULT_CLICKED_ARTICLES_COL,
    DEFAULT_IMPRESSION_ID_COL,
]
HISTORY_SIZE = 10
FRACTION = 0.01

df_train = (
    ebnerd_from_path(PATH.joinpath(DATASPLIT, "train"), history_size=HISTORY_SIZE)
    .select(COLUMNS)
```

（4）本段代码定义了 ebnerd_from_path_test 函数，用于处理和加载测试数据。其中 df_raw_history.select()函数可以从 df_raw_history 数据中选择用户列和历史文章 ID 列，使用 truncate_history 函数截断历史记录，截断大小由 history_size 决定。df_raw_behavior.pipe()函数用来将 df_raw_behavior 数据和经过处理的 df_history 数据左连接，连接键为用户列。

```
COLUMNS = [
    DEFAULT_USER_COL,
    DEFAULT_HISTORY_ARTICLE_ID_COL,
    DEFAULT_INVIEW_ARTICLES_COL,
    DEFAULT_CLICKED_ARTICLES_COL,
    DEFAULT_IMPRESSION_ID_COL,
]
HISTORY_SIZE = 10
FRACTION = 0.01
def ebnerd_from_path_test(df_raw_history: pl.DataFrame, df_raw_behavior: pl.DataFrame, history_size: int =
    """
    Load ebnerd (test) - function
    """
    df_history = (
        df_raw_history
        .select(DEFAULT_USER_COL, DEFAULT_HISTORY_ARTICLE_ID_COL)
        .pipe(
            truncate_history,
            column=DEFAULT_HISTORY_ARTICLE_ID_COL,
            history_size=history_size,
            padding_value=0,
            enable_warning=False,
        )
    )
    df_behaviors = (
        df_raw_behavior
        .pipe(
            slice_join_dataframes,
            df2=df_history,
            on=DEFAULT_USER_COL,
            how="left",
        )
    )
    return df_behaviors
```

（5）由于原始代码中没有涉及处理 test 集的部分，也没有引入 test 集。本段代码用来导入 test 集中的 history.parquet 数据和 behaviors.parquet 数据并进行处理。

```
df_test_history = pl.read_parquet(PATH.joinpath(DATASPLIT,"test/history.parquet"))
df_test_behavior = pl.read_parquet(PATH.joinpath(DATASPLIT,"test/behaviors.parquet"))
df_test_behavior = df_test_behavior.with_columns(
    pl.Series("article_ids_clicked", [None], dtype = pl.List(pl.Int32))
)
df_test_behavior.head(2)
```

（6）导入 test 数据集中的 history.parquet 数据和 behaviors.parquet 数据后，定义 df_test 数据集，作为测试数据。

```
df_test = (
    ebnerd_from_path_test(df_test_history, df_test_behavior, history_size=HISTORY_SIZE)
    .select(COLUMNS)
    .pipe(create_binary_labels_column)
)

df_test.head(2)
```

（7）本段为加载 article.parquet 数据（实际上未使用）

```
Load articles

df_articles = pl.read_parquet(PATH.joinpath("ebnerd_small/articles.parquet"))
df_articles.head(2)
```

Python

（8）本段代码使用了 HuggingFace’s tokenizer。通过加载 FacebookAI/xlm-roberta-base 模型以及分词器，完成了对文章数据的处理，并将文章标题转换为词嵌入，然后创建了从文章 ID 到词嵌入的映射。

需要注意的是，加载 FacebookAI/xlm-roberta-base 模型需要运行代码时下载，如果不使用镜像，可能造成网络延迟错误。

Init model using HuggingFace's tokenizer and wordembedding

In the original implementation, they use the GloVe embeddings and tokenizer. To get going fast, we'll use a multilingual LLM from Hugging Face. Utilizing the tokenizer to tokenize the articles and the word-embedding to init NRMS.

```
TRANSFORMER_MODEL_NAME = "FacebookAI/xlm-roberta-base"
TEXT_COLUMNS_TO_USE = [DEFAULT_SUBTITLE_COL, DEFAULT_TITLE_COL]
MAX_TITLE_LENGTH = 30

# LOAD HUGGINGFACE:
transformer_model = AutoModel.from_pretrained(TRANSFORMER_MODEL_NAME)
transformer_tokenizer = AutoTokenizer.from_pretrained(TRANSFORMER_MODEL_NAME)

# We'll init the word embeddings using the
word2vec_embedding = get_transformers_word_embeddings(transformer_model)
#
df_articles, cat_cal = concat_str_columns(df_articles, columns=TEXT_COLUMNS_TO_USE)
df_articles, token_col_title = convert_text2encoding_with_transformers(
    df_articles, transformer_tokenizer, cat_cal, max_length=MAX_TITLE_LENGTH
)
# =>
article_mapping = create_article_id_to_value_mapping(
    df=df_articles, value_col=token_col_title
)
```

[8] Python

（9）在加载完数据后，我们构建完成了 train，validation，test 数据集。本段代码完成对三个测试集的初始化。

其中重要的是 batch_size 可以调整一次性参与训练或者测试的数据集样本量，batch_size 越大，参与训练或者预测的样本量越多，如果训练显卡性能足够，可以起到加速作用。

Initiate the dataloaders

In the implementations we have disconnected the models and data. Hence, you should built a dataloader that fits your needs.

```
train_dataloader = NRMSDataLoader(
    behaviors=df_train,
    article_dict=article_mapping,
    unknown_representation="zeros",
    history_column=DEFAULT_HISTORY_ARTICLE_ID_COL,
    eval_mode=False,
    batch_size=64,
)
val_dataloader = NRMSDataLoader(
    behaviors=df_validation,
    article_dict=article_mapping,
    unknown_representation="zeros",
    history_column=DEFAULT_HISTORY_ARTICLE_ID_COL,
    eval_mode=True,
    batch_size=32,
)
test_dataloader = NRMSDataLoader(
    behaviors=df_test,
    article_dict=article_mapping,
    unknown_representation="zeros",
    history_column=DEFAULT_HISTORY_ARTICLE_ID_COL,
    eval_mode=True,
    batch_size=32,
)
```

[9] Python

（10）本段代码为模型训练部分，使用 python 库中的 NRMS 训练函数完成对 NRMS 模型的训练。

Train the model

```
MODEL_NAME = "NRMS"
LOG_DIR = f"downloads/runs/{MODEL_NAME}"
MODEL_WEIGHTS = f"downloads/data/state_dict/{MODEL_NAME}/weights"

# CALLBACKS
tensorboard_callback = tf.keras.callbacks.TensorBoard(log_dir=LOG_DIR, histogram_freq=1)
early_stopping = tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=2)
modelcheckpoint = tf.keras.callbacks.ModelCheckpoint(
    filepath=MODEL_WEIGHTS, save_best_only=True, save_weights_only=True, verbose=1
)

hparams_nrms.history_size = HISTORY_SIZE
model = NRMSModel(
    hparams=hparams_nrms,
    word2vec_embedding=word2vec_embedding,
    seed=42,
)
hist = model.model.fit(
    train_dataloader,
    validation_data=val_dataloader,
    epochs=1,
    callbacks=[tensorboard_callback, early_stopping, modelcheckpoint],
)
_ = model.model.load_weights(filepath=MODEL_WEIGHTS)
```

[10] Python

（11）本段代码为训练完模型后，使用模型完成实际测试集预测部分。

Example how to compute some metrics:

```
#pred_validation = model.scorer.predict(val_dataloader)

pred_test = model.scorer.predict(test_dataloader)
```

Pytho

（12）下面两段代码完成预测后的打标签工作。

Add the predictions to the dataframe:

```
# df_validation = add_prediction_scores(df_validation, pred_validation.tolist()).pipe(
#     add_known_user_column, known_users=df_train[DEFAULT_USER_COL]
# )
# df_validation.head(2)
```

Python

```
df_test = add_prediction_scores(df_test, pred_test.tolist()).pipe(
 add_known_user_column, known_users=df_train[DEFAULT_USER_COL]
)
df_test.head(2)
```

Python

Compute metrics

```
# '''metrics = MetricEvaluator(
#     labels=df_test["labels"].to_list(),
#     predictions=df_test["scores"].to_list(),
#     metric_functions=[AucScore(), MrrScore(), NdcgScore(k=5), NdcgScore(k=10)],
# )
# metrics.evaluate()'''
```

Python

```
metrics = MetricEvaluator(
    labels=df_validation["labels"].to_list(),
    predictions=df_validation["scores"].to_list(),
    metric_functions=[AucScore(), MrrScore(), NdcgScore(k=5), NdcgScore(k=10)],
)
metrics.evaluate()
```

Python

（13）本段代码在 df_test 数据框中添加一个新的列 rank_scores，其值通过对 scores 列的每个元素通过 rank_predictions_by_scores 函数映射得到。完成分数预测。 最终将结果写入到本地 downloads/predictions.txt 文件中。从而完成了对测试集的预测。

Make submission file

```
'''df_validation = df_validation.with_columns(
    pl.col("scores")
    .map_elements(lambda x: list(rank_predictions_by_score(x)))
    .alias("ranked_scores")
)
df_validation.head(2)'''
df_test = df_test.with_columns(
    pl.col("scores")
    .map_elements(lambda x: list(rank_predictions_by_score(x)))
    .alias("ranked_scores")
)
df_test.head(2)
```

Python

This is using the validation, simply add the testset to your flow.

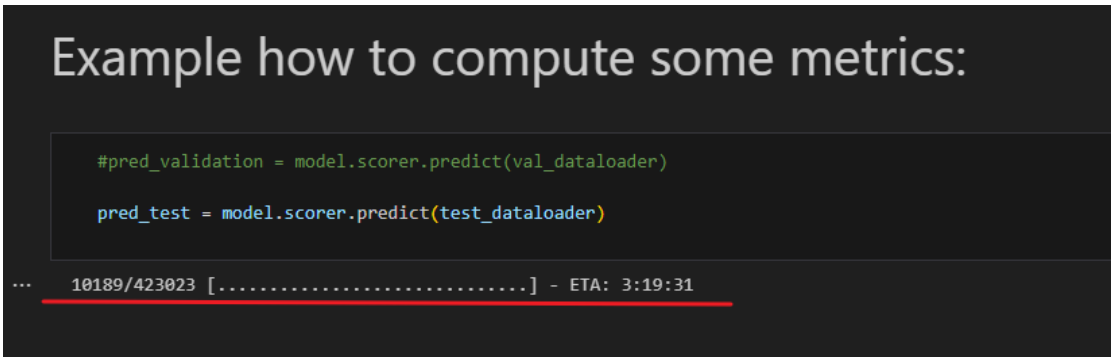
```
write_submission_file(
    impression_ids=df_test[DEFAULT_IMPRESSION_ID_COL],
    prediction_scores=df_test["ranked_scores"],
    path="downloads/predictions.txt",
)
```

Python

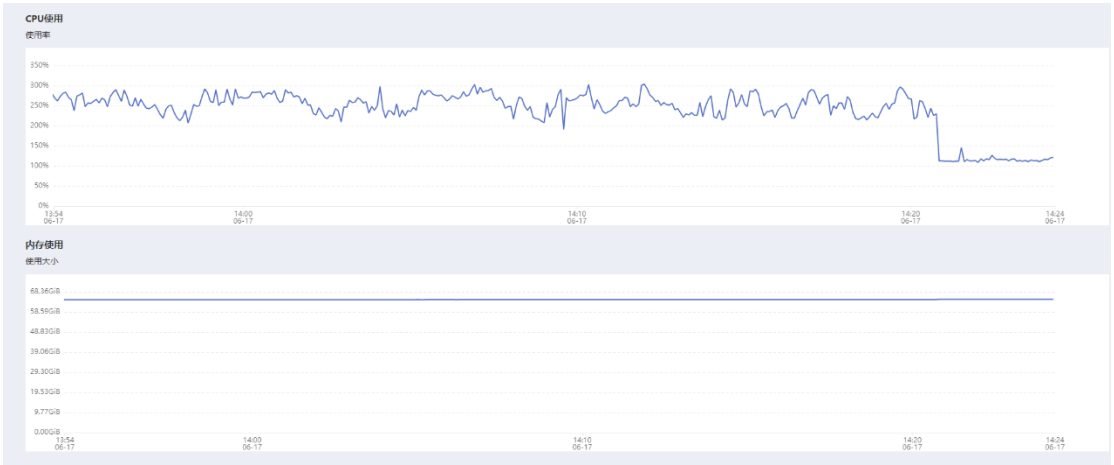
4、运行中间状况以及运行模型时遇到的问题

总的来说，在运行过程中主要出现了运行内存不足以及运行时间过长两个问题。

（1）第一个问题出现在内存占用上，本机 16G 内存远远不足。模型可以正常训练完成，但是在预测测试集时总共 423023 条（batch_size 调整为 32）只能预测出 10189，不足 1/40。预测状况如下图：



（2）使用云服务器可以在预测阶段避免内存不足现象，从而完成 423023 个预测。



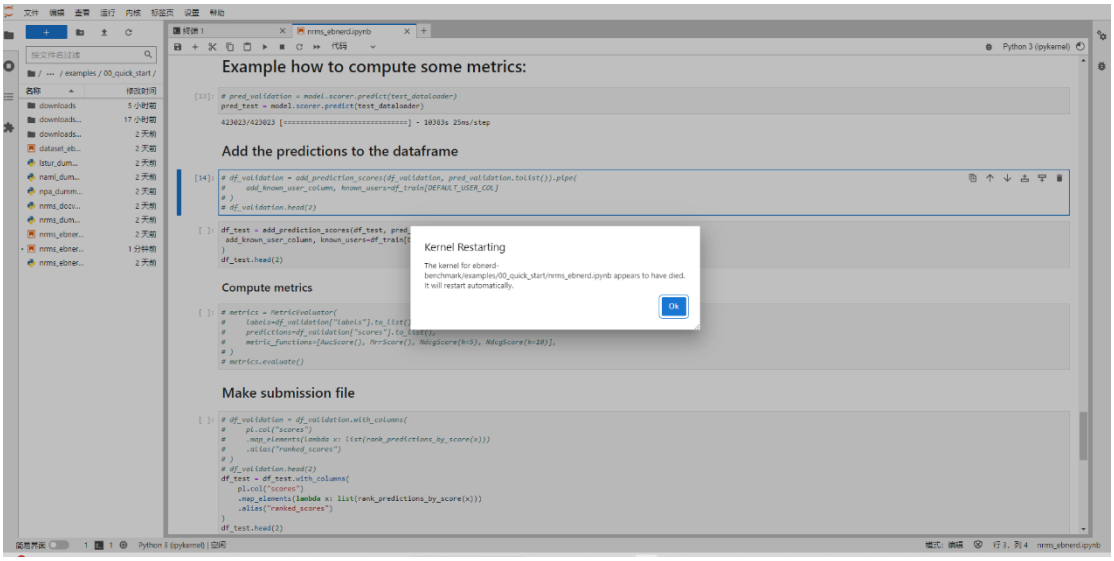
可以看到，在 prediction 阶段，内存最大占用达到接近 68G。

（3）但是在 Add the predictions to the dataframe 阶段。同样出现了内存不足情况。200G 内存也会触顶，导致操作系统为了维护稳定性，不得已将最大占用内存进程杀死，也就是将模型预测进行杀死。

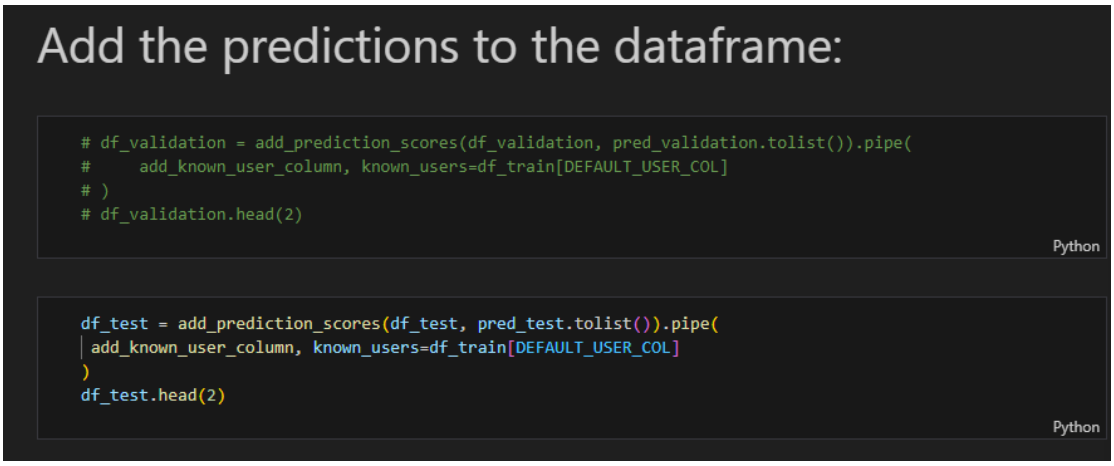


本图中实际开辟最大上限内存为 200G，当内存占用达到 195G 左右时，内存断崖式下跌，意味着内存占用被瞬间清空，进程被操作系统杀死。

操作系统杀死该进程后，内核错误，模型运行失败。



该问题发生在下图所示代码执行阶段。



（4）经过测试，发现内存最大触顶为 205G 左右，开辟 300G 内存空间解决内存问题。

（5）使用 L20 显卡进行模型构建以及测试集预测，发现最终模型预测时间超过 30 个小时。在 10 个小时的执行过程中，我发现投入资金和精力耗费过大，我终止了模型预测，得到了 1/3 的预测数据结果。

由下图可以看到，每秒运行速度为 121 条左右，经过判断，一共有 1200 万条数据。计算所得，需要至少 27 个小时才能完成预测。耗费时间和精力过大，在我完成 4197713 条数据预测时，结束了预测，得到了大小为 155.7MB 大小的预测数据集。

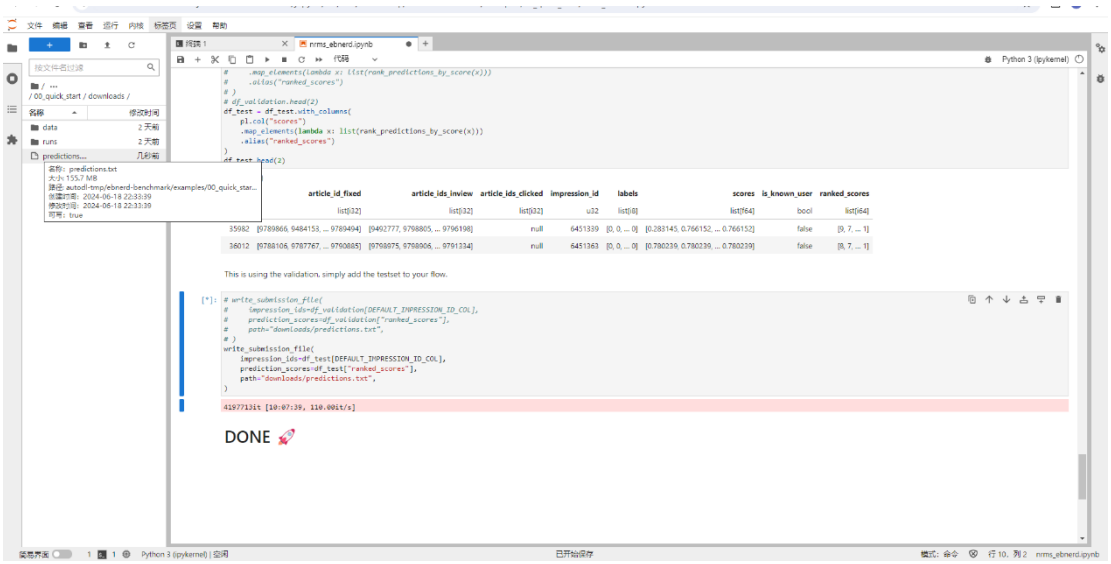
This is using the validation, simply add the testset to your flow.

```
[*]: # write_submission_file(
     #     impression_ids=df_validation[DEFAULT_IMPRESSION_ID_COL],
     #     prediction_scores=df_validation["ranked_scores"],
     #     path="downloads/predictions.txt",
     # )
     write_submission_file(
         impression_ids=df_test[DEFAULT_IMPRESSION_ID_COL],
         prediction_scores=df_test["ranked_scores"],
         path="downloads/predictions.txt",
     )
```

```
1337692it [3:01:31, 121.58it/s]
```

DONE

最终预测结果（400 万条数据，总数据的 1/3）如下图



特别的，在 prediction 过程中，发现最后 10000 个速度格外慢，打开 parquet 文件查看后发现是较大数据。

Example how to compute some metrics:

```
[*]: # pred_validation = model.scorer.predict(test_dataloader)
     pred_test = model.scorer.predict(test_dataloader)
```

```
418178/423023 [============================>.] - ETA: 1:36
```

6、运行结果

因为提交需要提交一个完整的预测结果，所以我选择将预测出的 1/3 结果与随机样本结果进行组合上传提交。即将随机预测样本结果去除前 1/3，我的 1/3 预测结果贴合到随机预测样本结果上上传提交。

下两图为随机预测提交结果：



Ranking Metrics: Grouped by Selected Dates

	AUC*	MRR*	NDCG5*	NDCG10*
2023-06-01	0.4995	0.3160	0.3495	0.4345
2023-06-02	0.4999	0.3160	0.3499	0.4344
2023-06-03	0.4998	0.3139	0.3465	0.4310
2023-06-04	0.5000	0.3134	0.3458	0.4303
2023-06-05	0.4997	0.3177	0.3523	0.4362
2023-06-06	0.4999	0.3129	0.3446	0.4303
2023-06-07	0.5002	0.3190	0.3538	0.4391
2023-06-08	0.4991	0.3168	0.3498	0.4370
MEAN	0.4998	0.3157	0.3490	0.4341

'*' 50% of the testset

下两图为结合了 1/3 模型预测结果的提交结果

71063	predictions619.zip	2024-06-19 02:56	Finished	0.50	

Ranking Metrics: Grouped by Selected Dates

	AUC*	MRR*	NDCG5*	NDCG10*
2023-06-01	0.4992	0.3158	0.3495	0.4344
2023-06-02	0.5002	0.3161	0.3504	0.4346
2023-06-03	0.4998	0.3141	0.3469	0.4312
2023-06-04	0.4999	0.3132	0.3455	0.4303
2023-06-05	0.4997	0.3175	0.3522	0.4361
2023-06-06	0.4996	0.3124	0.3442	0.4297
2023-06-07	0.5002	0.3190	0.3535	0.4390
2023-06-08	0.4989	0.3169	0.3497	0.4368
MEAN	0.4997	0.3156	0.3490	0.4340

'*' 50% of the testset

结果分析与体会：

可以看到，结合了 1/3 模型预测的提交结果得分依然为 0.5，在 Ranking Metrics 中，AUC，MRR*,NDCG5*等不同指标数据在 2023 年 6 月第一周大部分天数与随机预测的不相同，有质量更好的情况，也有质量更坏的情况。

推测原因，是因为训练测试集的数据集太小了，small 数据集中 behaviors.parquet 数据集只有不到 10M，history.parquet 数据集只有 22M。过于小的数据集不能够抓住模型的实际规律，导致了欠拟合。推测使用 large 数据集可以改善这一个状况，但是实际训练过程中，large 数据集加载就需要非常长的时间，实际训练无法完成。